

Tower Of Hanoi

–Space Complexity(Detailed Analysis)

Program:

```
// Tower of Hanoi

#include <iostream>
using namespace std;

void toh(int n, char src, char dest, char aux)
{
    if (n == 0)
    {
        return ;
    }

    toh(n - 1, src, aux, dest);

    cout << "Move " << n << " from " << src << " to " <<
dest << endl;

    toh(n - 1, aux, dest, src);
}

int main()
{
    toh(3, 'A', 'C', 'B');
    return 0;
}
```

Space Complexity

1. Input Space Complexity: $O(1)$

Just like the factorial example, the Input Space Complexity is the memory dedicated strictly to the initial variables your algorithm needs to start.

- **Parametric Input Space ($O(1)$):** Looking at the function signature: `void toh(int n, char src, char dest, char aux)`. Every single time this function is called, it accepts four specific variables: an integer (n , usually 4 bytes) and three characters (src , $dest$, aux , 1 byte each). Whether user moving 3 disks or 3,000 disks, these four variables take up the exact same tiny, constant amount of memory. Thus, the parametric space is $O(1)$.

- **Holistic Input Space ($O(1)$):** What is the overall data structure being processed? Unlike sorting an array of 1,000 items (where the array itself takes up memory proportional to n), the Tower of Hanoi only requires knowing the number of disks, n . The entire problem is defined by a single integer in `main()`. Therefore, the holistic input space is strictly constant, $O(1)$.

Input Space Complexity = $O(1)$

Auxiliary Space

Auxiliary space = extra memory used during execution (excluding input storage)

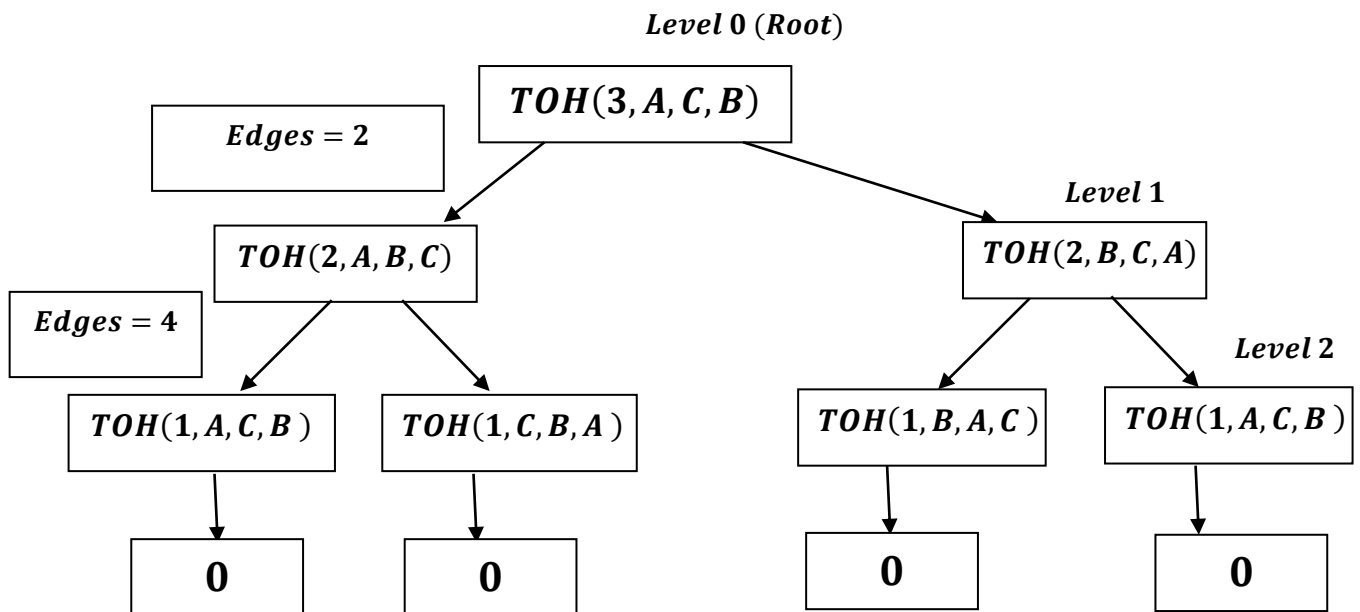
Each stack frame stores:

- n
- $src, dest, aux$
- $return\ address$

Each frame = $O(1)$ space per recursive call

Now, let's visualize the recursive tree:

for $n = 3$:

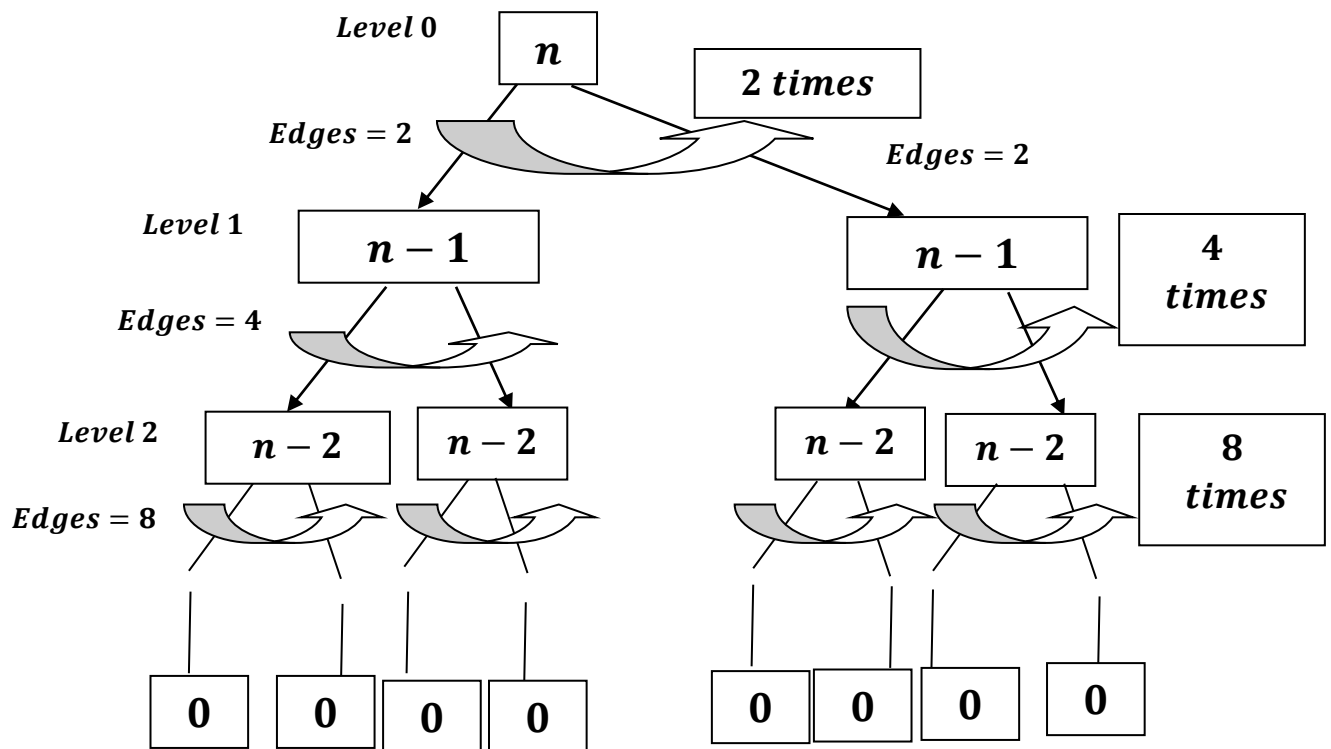


$$\text{Nodes}(N) = \text{Edges}(6) + 1(\text{Root}) = 7.$$

Depth = $TOH(3, A, C, B) \rightarrow TOH(2, A, B, C) \rightarrow TOH(1, A, C, B)$

No. of edges = 2 = Depth.

For `n`:



Note TOH(Tower of Hanoi) is conceptually perfect binary tree, therefore at level `k` = No. of nodes = 2^k

We know, Edges = Nodes – 1

Here is the step – by – step mathematical breakdown:

1. **Level 0 (The Root):** The tree starts at the very top with the initial input value of n .
2. **Level 1:** The algorithm branches, and 1 is subtracted from the input. The nodes here hold the value $n - 1$.
3. **Level 2:** The algorithm branches again, subtracting another 1. The nodes hold the value $n - 2$.

4. **Level k (The Leaves)**: This pattern of subtracting 1 continues downwards until we reach the bottom of the tree (the base cases), where the nodes hold the value 0. Let's call this final depth level k .

To find the actual depth (k), we set the formula for the value at level k equal to the value of the base case [$TOH(0)$]:

$$n - k = 0$$

Solving for k :

$$k = n$$

Conclusion:

The depth of the tree (the number of edges or steps from the root down to the farthest leaf) is exactly n .

We got,

$$n - k = 0 [\text{user input `n`} - \text{Edges}(k) = 0]$$

$$\Rightarrow k = n (\text{This is height based})$$

Though, $k = n$, but Levels actually in TOH goes from $0, 1, \dots, N - 1$ as $TOH(1)$ is the leaf node.

$$\therefore k = n - 1 (\text{Based on Maximum Level Index})$$

Therefore ,total nodes for level k =

$$2^0 + 2^1 + 2^2 + \dots + 2^{n-1}$$

By geometric series:

$$S(n) = ar^0 + ar^1 + ar^2 + \dots + ar^n = a \left(\frac{r^{n+1} - 1}{r - 1} \right), \text{ where}$$

S(n) is sum of first n terms , a is coefficient and r is common ratio between the consecutive terms.

$$\Rightarrow 1 \left(\frac{2^{n-1+1} - 1}{2 - 1} \right)$$

$$\Rightarrow 2^n - 1 \text{ nodes.}$$

∴ Entire recursion tree will have $(2^n - 1) - 1$ edges.

Total number of Edges = $2^n - 2$ edges.

For Depth Calculation

But for depth, we take $edge(k) = n$.

$$***$N(\text{Nodes}) = \text{Edges}(k) + 1(\text{Root})$***$$

$$***$\Rightarrow N = n + 1$*** ,$$

$$***\text{Maximum Depth} = \text{Number of Edges} = n.***$$

Based on Active stackframe:

$\therefore$ Total Auxiliary Space

$$***= (\text{Maximum Depth} + 1) \times \left(\begin{array}{c} \text{Space per} \\ \text{Recursive Call} \end{array} \right)***$$

$\left[\text{Here} + 1 \text{ for Root Node , which is also an active} \right]$
stack frame

$$***= \Theta(n + 1) \times \Theta(1)***$$

$$***= \Theta(n) \times \Theta(1)***$$

$$***= \Theta(n)***$$

∴ Total Space Complexity:

$$\begin{aligned} \text{Input Space} + \text{Auxiliary Space} &= \Theta(1) + \Theta(n) \\ &= \Theta(n). \end{aligned}$$
